

Mathematical Analysis of Simulated GPU Clusters

A Comprehensive Guide to Software-Based Supercomputing

Abstract

This document presents a mathematical framework for understanding simulated GPU clusters, where virtual hardware components bypass physical constraints to achieve supercomputer-level performance. Through precise mathematical modeling and code implementations, we demonstrate how software simulations can theoretically outperform physical hardware by eliminating overhead factors.

1. Introduction and Theoretical Foundation

1.1 Core Concept

Simulated GPU clusters operate on the principle of **virtual resource multiplication**, where:

- Physical constraints (power, heat, latency) are eliminated
- Computational operations exist purely in software space
- Resource scaling becomes mathematically unbounded

1.2 Mathematical Definition

Let S represent a simulated GPU cluster:

$$S = \{G_1, G_2, \dots, G_n\} \text{ where } n \rightarrow \infty$$

Each simulated GPU G_i is defined by:

$$G_i = \{P, M, O\} \text{ where:}$$

- P = Processing capability (FLOPS)
- M = Memory bandwidth
- O = Operations queue

2. Performance Modeling

2.1 Theoretical Peak Performance

For a simulated cluster of n GPUs, each with performance p TFLOPS:

$$\text{Total Performance} = n \times p \text{ TFLOPS}$$

Real-world example:

- Nvidia H100: 141 TFLOPS
- 50,000,000 simulated H100s: 7,050,000,000 TFLOPS = 7.05 EFLOPS

2.2 Efficiency Factor

In physical systems, efficiency (η) is typically 0.6-0.8 due to:

- Interconnect latency
- Power throttling
- Thermal constraints

In simulated systems: $\eta \approx 1.0$ (theoretical maximum)

2.3 Code Implementation: Performance Calculator

python

```
class SimulatedGPUCluster:
    def __init__(self, gpu_count, gpu_performance_tflops, efficiency=1.0):
        self.gpu_count = gpu_count
        self.gpu_performance = gpu_performance_tflops
        self.efficiency = efficiency

    def total_performance(self):
        """Calculate total cluster performance in TFLOPS"""
        return self.gpu_count * self.gpu_performance * self.efficiency

    def performance_in_exaflops(self):
        """Convert to ExaFLOPS"""
        return self.total_performance() / 1_000_000

    def cost_comparison(self, physical_gpu_cost_usd, power_cost_per_kwh, power_draw_kw):
        """Compare costs with physical implementation"""
        physical_cost = self.gpu_count * physical_gpu_cost_usd
        # Power cost for one year of operation
        annual_power_cost = self.gpu_count * power_draw_kw * 24 * 365 * power_cost_per_kwh
        return {
            'hardware_cost': physical_cost,
            'annual_power_cost': annual_power_cost,
            'total_first_year': physical_cost + annual_power_cost,
            'simulated_cost': 0 # Negligible
        }

# Example usage
cluster = SimulatedGPUCluster(50_000_000, 141) # 50M H100s
print(f"Total Performance: {cluster.performance_in_exaflops():.2f} ExaFLOPS")
print(f"Cost Analysis: {cluster.cost_comparison(30000, 0.10, 0.7)}")
```

3. Daisy-Chain Processing Architecture

3.1 Mathematical Model

Data flows through a daisy-chain of n GPUs, where each GPU processes data and passes results:

Input $\rightarrow G_1 \rightarrow G_2 \rightarrow \dots \rightarrow G_n \rightarrow$ Output

Processing time for sequential architecture:

$$T_{\text{sequential}} = \sum_{i=1}^n t_{\text{process}_i} + \sum_{i=1}^{n-1} t_{\text{transfer}_i}$$

In simulated systems: $t_{\text{transfer}_i} \approx 0$

Therefore: $T_{\text{simulated}} \approx \max(t_{\text{process}_i})$ (parallel simulation)

3.2 Code Implementation: Daisy-Chain Simulator

python

```

import numpy as np
from concurrent.futures import ThreadPoolExecutor
import time

class GPUSimulator:
    def __init__(self, gpu_id, processing_power_tflops):
        self.gpu_id = gpu_id
        self.processing_power = processing_power_tflops
        self.operations_completed = 0

    def process_data(self, data_batch, operation_count):
        """Simulate GPU processing of a data batch"""
        # Simulate computation time based on operations and processing power
        computation_time = operation_count / (self.processing_power * 1e12)

        # Actual processing (matrix operations as example)
        result = np.matmul(data_batch, data_batch.T)
        self.operations_completed += operation_count

        return result, computation_time

class DaisyChainSimulator:
    def __init__(self, gpu_specs):
        """
        gpu_specs: list of tuples (gpu_id, processing_power_tflops)
        """
        self.gpus = [GPUSimulator(gpu_id, power) for gpu_id, power in gpu_specs]
        self.total_gpus = len(self.gpus)

    def process_sequential(self, initial_data, operations_per_gpu):
        """Process data through the daisy chain sequentially"""
        current_data = initial_data
        total_time = 0

        for gpu in self.gpus:
            result, proc_time = gpu.process_data(current_data, operations_per_gpu)
            current_data = result
            total_time += proc_time

        return current_data, total_time

    def process_parallel_simulation(self, initial_data, operations_per_gpu):
        """Simulate parallel processing (instant transfer)"""

```

```

start_time = time.time()

# In a true simulation, we can process stages simultaneously
with ThreadPoolExecutor(max_workers=self.total_gpus) as executor:
    futures = []
    for i, gpu in enumerate(self.gpus):
        # Each GPU processes a part of the workload
        futures.append(executor.submit(gpu.process_data, initial_data, operations_per_g

    results = [future.result() for future in futures]

end_time = time.time()
return results, end_time - start_time

def get_total_performance(self):
    """Calculate total cluster performance"""
    return sum(gpu.processing_power for gpu in self.gpus)

# Example usage
gpu_specs = [(i, 141) for i in range(10000)] # 10K H100 GPUs
daisy_chain = DaisyChainSimulator(gpu_specs)

# Generate test data
test_data = np.random.rand(1000, 1000)
operations = 1e15 # 1 PetaFLOP of operations

# Compare sequential vs simulated parallel
seq_result, seq_time = daisy_chain.process_sequential(test_data, operations)
par_results, par_time = daisy_chain.process_parallel_simulation(test_data, operations)

print(f"Sequential processing time: {seq_time:.6f} seconds")
print(f"Parallel simulation time: {par_time:.6f} seconds")
print(f"Speedup factor: {seq_time / par_time:.2f}x")
print(f"Total cluster performance: {daisy_chain.get_total_performance():.2f} TFLOPS")

```

4. Elimination of Physical Constraints

4.1 Mathematical Analysis of Overhead Reduction

Physical GPU clusters face several constraints:

1. **Power Constraint:** $P_{\text{total}} = \sum P_i \leq P_{\text{max}}$
2. **Thermal Constraint:** $T_i = f(P_i, \text{cooling_rate}) \leq T_{\text{max}}$

3. **Bandwidth Constraint:** $B_{\text{effective}} = B_{\text{peak}} \times \text{efficiency_factor}$

4. **Latency Constraint:** $L_{\text{total}} = \sum L_{\text{interconnect}} + L_{\text{memory}}$

In simulated systems:

- $P_{\text{total}} \rightarrow 0$ (no real power consumption)
- $T_i \rightarrow T_{\text{ambient}}$ (no heat generation)
- $B_{\text{effective}} \rightarrow B_{\text{theoretical}}$ (no physical limitations)
- $L_{\text{total}} \rightarrow 0$ (instant data transfer)

4.2 Code Implementation: Constraint Analysis

python

```

import matplotlib.pyplot as plt
import numpy as np

class ConstraintAnalyzer:
    def __init__(self):
        self.physical_constraints = {}
        self.simulated_constraints = {}

    def calculate_power_consumption(self, gpu_count, power_per_gpu_kw):
        """Calculate power consumption for physical vs simulated"""
        physical_power = gpu_count * power_per_gpu_kw
        simulated_power = 0.0001 # Negligible host machine power

        return {
            'physical': physical_power,
            'simulated': simulated_power,
            'reduction_factor': physical_power / simulated_power
        }

    def calculate_thermal_load(self, gpu_count, heat_per_gpu_kw):
        """Calculate thermal load"""
        physical_heat = gpu_count * heat_per_gpu_kw
        simulated_heat = 0 # No heat generation

        return {
            'physical': physical_heat,
            'simulated': simulated_heat,
            'cooling_capacity_needed': physical_heat * 1.2 # 20% overhead
        }

    def calculate_interconnect_latency(self, gpu_count, latency_per_hop_ns):
        """Calculate network latency in multi-GPU setup"""
        # Simplified model: average hops between GPUs
        avg_hops = np.sqrt(gpu_count)
        physical_latency = avg_hops * latency_per_hop_ns
        simulated_latency = 0 # No physical interconnect

        return {
            'physical': physical_latency,
            'simulated': simulated_latency,
            'improvement_factor': 'infinite'
        }

```

```

def generate_constraint_report(self, gpu_counts):
    """Generate comprehensive constraint analysis"""
    results = {
        'gpu_counts': gpu_counts,
        'power_consumption': [],
        'thermal_load': [],
        'latency': []
    }

    for count in gpu_counts:
        power = self.calculate_power_consumption(count, 0.7) # 700W per H100
        thermal = self.calculate_thermal_load(count, 0.7)
        latency = self.calculate_interconnect_latency(count, 100) # 100ns

        results['power_consumption'].append(power)
        results['thermal_load'].append(thermal)
        results['latency'].append(latency)

    return results

def plot_constraints(self, results):
    """Visualize constraint comparisons"""
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 10))

    # Power Consumption
    physical_power = [r['physical'] for r in results['power_consumption']]
    simulated_power = [r['simulated'] for r in results['power_consumption']]

    ax1.loglog(results['gpu_counts'], physical_power, 'r-', label='Physical')
    ax1.loglog(results['gpu_counts'], simulated_power, 'b--', label='Simulated')
    ax1.set_xlabel('Number of GPUs')
    ax1.set_ylabel('Power Consumption (kW)')
    ax1.set_title('Power Consumption Comparison')
    ax1.legend()
    ax1.grid(True)

    # Thermal Load
    physical_heat = [r['physical'] for r in results['thermal_load']]
    cooling_needed = [r['cooling_capacity_needed'] for r in results['thermal_load']]

    ax2.loglog(results['gpu_counts'], physical_heat, 'r-', label='Heat Generated')
    ax2.loglog(results['gpu_counts'], cooling_needed, 'orange', label='Cooling Required')
    ax2.axhline(y=0, color='b', linestyle='--', label='Simulated (zero)')
    ax2.set_xlabel('Number of GPUs')

```

```

ax2.set_ylabel('Thermal Load (kW)')
ax2.set_title('Thermal Load Comparison')
ax2.legend()
ax2.grid(True)

# Latency
physical_latency = [r['physical'] for r in results['latency']]

ax3.semilogx(results['gpu_counts'], physical_latency, 'r-', label='Physical')
ax3.axhline(y=0, color='b', linestyle='--', label='Simulated (zero)')
ax3.set_xlabel('Number of GPUs')
ax3.set_ylabel('Latency (ns)')
ax3.set_title('Interconnect Latency Comparison')
ax3.legend()
ax3.grid(True)

plt.tight_layout()
return fig

# Usage example
analyzer = ConstraintAnalyzer()
gpu_counts = [1000, 10000, 100000, 1000000, 10000000, 50000000]
results = analyzer.generate_constraint_report(gpu_counts)
fig = analyzer.plot_constraints(results)
plt.show()

# Print summary
print("Constraint Analysis Summary:")
for i, count in enumerate(gpu_counts):
    print(f"\nGPU Count: {count:,}")
    print(f"Power Reduction: {results['power_consumption'][i]['reduction_factor']:.1e}x")
    print(f"Thermal Elimination: 100%")
    print(f"Latency Improvement: Infinite")

```

5. AI Acceleration Through Simulation

5.1 Mathematical Model of AI Speedup

When an AI system receives output from a simulated GPU cluster, the effective computational boost can be modeled as:

$$\text{Speedup_factor} = (\text{Simulated_FLOPS} \times \text{Efficiency_simulated}) / (\text{Real_FLOPS} \times \text{Efficiency_real})$$

Where:

- $\text{Efficiency_simulated} \approx 1.0$
- $\text{Efficiency_real} \approx 0.7$

For a 50M GPU simulation:

$$\text{Speedup} = (50\text{M} \times 141 \text{ TFLOPS} \times 1.0) / (1 \times 141 \text{ TFLOPS} \times 0.7) \approx 71.4 \text{ Million times}$$

5.2 Code Implementation: AI Acceleration Model

python

```

import time
import numpy as np
from typing import Dict, List, Tuple

class AIAccelerationModel:
    def __init__(self, base_ai_performance_tflops: float):
        self.base_performance = base_ai_performance_tflops
        self.simulation_overhead = 0.01 # 1% overhead for simulation

    def calculate_effective_speedup(self,
                                    simulated_gpus: int,
                                    gpu_performance_tflops: float,
                                    physical_efficiency: float = 0.7,
                                    simulated_efficiency: float = 0.98) -> Dict:
        """Calculate effective AI speedup from simulated cluster"""

        # Real cluster performance
        real_total_flops = simulated_gpus * gpu_performance_tflops * physical_efficiency

        # Simulated cluster performance
        sim_total_flops = simulated_gpus * gpu_performance_tflops * simulated_efficiency

        # AI acceleration factor
        acceleration_factor = sim_total_flops / self.base_performance

        # Effective speedup considering the AI can utilize the full output instantly
        effective_speedup = acceleration_factor * (1 - self.simulation_overhead)

        return {
            'real_cluster_performance': real_total_flops,
            'simulated_cluster_performance': sim_total_flops,
            'acceleration_factor': acceleration_factor,
            'effective_speedup': effective_speedup,
            'performance_gain_percentage': (effective_speedup - 1) * 100
        }

    def model_ai_inference_time(self,
                                task_complexity_flops: float,
                                use_simulation: bool = True) -> Dict:
        """Model AI inference time with and without simulation"""

        if use_simulation:
            # With 50M simulated GPUs

```

```

        simulated_cluster_perf = 50_000_000 * 141 * 0.98 # TFLOPS
        inference_time = task_complexity_flops / (simulated_cluster_perf * 1e12)
    else:
        # With base AI performance
        inference_time = task_complexity_flops / (self.base_performance * 1e12)

    return {
        'inference_time_seconds': inference_time,
        'inference_time_ms': inference_time * 1000,
        'effective_performance_tflops': task_complexity_flops / (inference_time * 1e12)
    }

def comparative_analysis(self, task_complexities: List[float]) -> Dict:
    """Compare performance across different task complexities"""
    results = {
        'task_flops': task_complexities,
        'base_ai_times': [],
        'simulated_times': [],
        'speedup_factors': []
    }

    for task_flops in task_complexities:
        base_result = self.model_ai_inference_time(task_flops, use_simulation=False)
        sim_result = self.model_ai_inference_time(task_flops, use_simulation=True)

        speedup = base_result['inference_time_seconds'] / sim_result['inference_time_seconds']

        results['base_ai_times'].append(base_result['inference_time_seconds'])
        results['simulated_times'].append(sim_result['inference_time_seconds'])
        results['speedup_factors'].append(speedup)

    return results

class RealTimeSimulation:
    def __init__(self, cluster_size: int, gpu_performance: float):
        self.cluster_size = cluster_size
        self.gpu_performance = gpu_performance
        self.total_performance = cluster_size * gpu_performance

    def simulate_real_time_processing(self, data_stream: List[np.ndarray],
                                     operations_per_data: float) -> Dict:
        """Simulate real-time processing of a data stream"""
        start_time = time.time()
        results = []

```

```

for data_chunk in data_stream:
    # Simulate instantaneous processing due to simulated cluster
    processing_time = operations_per_data / (self.total_performance * 1e12)

    # In reality, this would be much more complex processing
    result = np.mean(data_chunk) # Simplified operation
    results.append(result)

    # Simulate the "instant return" behavior
    time.sleep(0.001) # Minimal overhead

end_time = time.time()

return {
    'total_time': end_time - start_time,
    'average_time_per_chunk': (end_time - start_time) / len(data_stream),
    'theoretical_performance_tflops': self.total_performance,
    'results': results
}

# Example usage
ai_model = AIAccelerationModel(base_ai_performance_tflops=100) # Base AI with 100 TFLOPS

# Calculate speedup for 50M simulated H100s
speedup_analysis = ai_model.calculate_effective_speedup(50_000_000, 141)
print("AI Acceleration Analysis:")
print(f"Effective Speedup: {speedup_analysis['effective_speedup']:.2e}x")
print(f"Performance Gain: {speedup_analysis['performance_gain_percentage']:.1f}%")

# Compare inference times for different task complexities
task_complexities = [1e15, 1e16, 1e17, 1e18, 1e19] # FLOPS
comparison = ai_model.comparative_analysis(task_complexities)

print("\nInference Time Comparison:")
for i, flops in enumerate(task_complexities):
    print(f"Task: {flops:.0e} FLOPS")
    print(f"  Base AI: {comparison['base_ai_times'][i]:.6f} seconds")
    print(f"  Simulated: {comparison['simulated_times'][i]:.9f} seconds")
    print(f"  Speedup: {comparison['speedup_factors'][i]:.2e}x")

# Real-time simulation demo
simulator = RealTimeSimulation(50_000_000, 141)
data_stream = [np.random.rand(1000, 1000) for _ in range(100)]

```

```

real_time_results = simulator.simulate_real_time_processing(data_stream, 1e12)

print(f"\nReal-time Processing Results:")
print(f"Total processing time: {real_time_results['total_time']:.4f} seconds")
print(f"Average time per chunk: {real_time_results['average_time_per_chunk']:.6f} seconds")
print(f"Processing rate: {len(data_stream) / real_time_results['total_time']:.1f} chunks/second")

```

6. Economic and Environmental Impact

6.1 Mathematical Cost Analysis

The cost differential between physical and simulated systems can be expressed as:

$$\text{Cost_difference} = (\text{Hardware_cost} + \text{Power_cost} + \text{Cooling_cost} + \text{Maintenance_cost}) - \text{Simulation_cost}$$

Where:

- $\text{Hardware_cost} = n \times \text{GPU_price}$
- $\text{Power_cost} = n \times \text{Power_per_GPU} \times \text{Hours} \times \text{Electricity_rate}$
- $\text{Cooling_cost} = \text{Power_cost} \times \text{Cooling_efficiency_factor}$
- $\text{Maintenance_cost} = \text{Hardware_cost} \times \text{Maintenance_rate_per_year}$
- $\text{Simulation_cost} \approx \text{Host_machine_cost} + \text{Host_power_cost}$

6.2 Environmental Impact Model

Carbon emissions reduction:

$$\text{CO2_reduction} = (n \times \text{GPU_power} \times \text{Carbon_intensity} \times \text{Hours}) - \text{Host_emissions}$$

Where Carbon_intensity is measured in kg CO₂/kWh

6.3 Code Implementation: Cost-Benefit Analysis

python

```

import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

class EconomicAnalysis:
    def __init__(self):
        self.gpu_price_usd = 30000 # H100 price
        self.power_per_gpu_kw = 0.7 # 700W
        self.electricity_rate_per_kwh = 0.10 # $0.10/kWh
        self.cooling_efficiency = 1.5 # PUE factor
        self.maintenance_rate_annual = 0.15 # 15% of hardware cost
        self.carbon_intensity_kg_co2_per_kwh = 0.5 # Average grid

    def calculate_physical_cluster_costs(self,
                                        gpu_count: int,
                                        operation_years: int) -> Dict:
        """Calculate total costs for physical GPU cluster"""

        # Hardware cost
        hardware_cost = gpu_count * self.gpu_price_usd

        # Annual power cost
        annual_power_kwh = gpu_count * self.power_per_gpu_kw * 24 * 365
        annual_power_cost = annual_power_kwh * self.electricity_rate_per_kwh

        # Annual cooling cost
        annual_cooling_cost = annual_power_cost * (self.cooling_efficiency - 1)

        # Annual maintenance cost
        annual_maintenance_cost = hardware_cost * self.maintenance_rate_annual

        # Total operational cost per year
        annual_operational_cost = annual_power_cost + annual_cooling_cost + annual_maintenance_

        # Total cost over operation period
        total_cost = hardware_cost + (annual_operational_cost * operation_years)

    return {
        'hardware_cost': hardware_cost,
        'annual_power_cost': annual_power_cost,
        'annual_cooling_cost': annual_cooling_cost,
        'annual_maintenance_cost': annual_maintenance_cost,
        'annual_operational_cost': annual_operational_cost,
    }

```

```

        'total_cost': total_cost,
        'cost_breakdown': {
            'hardware': hardware_cost / total_cost,
            'power': (annual_power_cost * operation_years) / total_cost,
            'cooling': (annual_cooling_cost * operation_years) / total_cost,
            'maintenance': (annual_maintenance_cost * operation_years) / total_cost
        }
    }
}

```

```

def calculate_simulated_cluster_costs(self, operation_years: int) -> Dict:
    """Calculate costs for simulated cluster (host machine)"""

    # Host machine cost (high-end workstation)
    host_machine_cost = 50000 # $50K for powerful host

    # Host power consumption (much lower)
    host_power_kw = 2.0 # 2kW total
    annual_host_power_cost = host_power_kw * 24 * 365 * self.electricity_rate_per_kwh

    # Software/license costs (if any)
    annual_software_cost = 5000 # Simulation software

    # Minimal maintenance
    annual_maintenance_cost = host_machine_cost * 0.05 # 5%

    total_cost = host_machine_cost + (annual_host_power_cost + annual_software_cost + annual_maintenance_cost)

    return {
        'hardware_cost': host_machine_cost,
        'annual_power_cost': annual_host_power_cost,
        'annual_software_cost': annual_software_cost,
        'annual_maintenance_cost': annual_maintenance_cost,
        'total_cost': total_cost
    }

```

```

def calculate_environmental_impact(self,
                                   gpu_count: int,
                                   operation_years: int) -> Dict:
    """Calculate environmental impact comparison"""

    # Physical cluster emissions
    annual_kwh_physical = gpu_count * self.power_per_gpu_kw * 24 * 365
    annual_kwh_physical_with_cooling = annual_kwh_physical * self.cooling_efficiency
    annual_co2_physical = annual_kwh_physical_with_cooling * self.carbon_intensity_kg_co2_per_kwh

```

```
total_co2_physical = annual_co2_physical * operation_years
```

```
# Simulated cluster emissions (host machine only)
```

```
annual_kwh_simulated = 2.0 * 24 * 365 # 2kW host
```

```
annual_co2_simulated = annual_kwh_simulated * self.carbon_intensity_kg_co2_per_kwh
```

```
total_co2_simulated = annual_co2_simulated * operation_years
```

```
# Reduction
```

```
co2_reduction = total_co2_physical - total_co2_simulated
```

```
co2_reduction_percentage = (co2_reduction / total_co2_physical) * 100
```

```
return {
```

```
    'physical_annual_co2_kg': annual_co2_physical,
```

```
    'simulated_annual_co2_kg': annual_co2_simulated,
```

```
    'total_co2_reduction_kg': co2_reduction,
```

```
    'co2_reduction_percentage': co2_reduction_percentage,
```

```
    'equivalent_trees_planted': co2_reduction / 22 # ~22kg CO2/tree/year
```

```
}
```

```
def generate_comprehensive_report(self, gpu_counts: List[int], years: int = 5) -> Dict:
```

```
    """Generate comprehensive economic and environmental analysis"""
```

```
    results = {
```

```
        'gpu_counts': gpu_counts,
```

```
        'physical_costs': [],
```

```
        'simulated_costs': [],
```

```
        'cost_savings': [],
```

```
        'environmental_impact': []
```

```
    }
```

```
    simulated_cost = self.calculate_simulated_cluster_costs(years)
```

```
    for gpu_count in gpu_counts:
```

```
        physical_costs = self.calculate_physical_cluster_costs(gpu_count, years)
```

```
        cost_saving = physical_costs['total_cost'] - simulated_cost['total_cost']
```

```
        cost_saving_percentage = (cost_saving / physical_costs['total_cost']) * 100
```

```
        env_impact = self.calculate_environmental_impact(gpu_count, years)
```

```
        results['physical_costs'].append(physical_costs)
```

```
        results['simulated_costs'].append(simulated_cost)
```

```
        results['cost_savings'].append({
```

```
            'absolute_saving': cost_saving,
```

```
            'percentage_saving': cost_saving_percentage
```

```

    })
    results['environmental_impact'].append(env_impact)

    return results

def create_visualization(self, analysis_results: Dict) -> plt.Figure:
    """Create comprehensive visualization of results"""

    fig = plt.figure(figsize=(15, 12))

    # Cost comparison
    ax1 = plt.subplot(2, 2, 1)
    gpu_counts = analysis_results['gpu_counts']
    physical_costs = [r['total_cost'] for r in analysis_results['physical_costs']]
    simulated_costs = [r['total_cost'] for r in analysis_results['simulated_costs']]

    ax1.loglog(gpu_counts, physical_costs, 'r-o', label='Physical Cluster')
    ax1.loglog(gpu_counts, simulated_costs, 'b--s', label='Simulated Cluster')
    ax1.set_xlabel('Number of GPUs')
    ax1.set_ylabel('Total Cost (USD)')
    ax1.set_title('Cost Comparison Over 5 Years')
    ax1.legend()
    ax1.grid(True)

    # Cost savings percentage
    ax2 = plt.subplot(2, 2, 2)
    savings_pct = [s['percentage_saving'] for s in analysis_results['cost_savings']]
    ax2.semilogx(gpu_counts, savings_pct, 'g-o')
    ax2.set_xlabel('Number of GPUs')
    ax2.set_ylabel('Cost Savings (%)')
    ax2.set_title('Cost Savings Percentage')
    ax2.grid(True)

    # Environmental impact
    ax3 = plt.subplot(2, 2, 3)
    co2_reduction = [e['total_co2_reduction_kg'] for e in analysis_results['environmental_i
    ax3.loglog(gpu_counts, co2_reduction, 'brown-^')
    ax3.set_xlabel('Number of GPUs')
    ax3.set_ylabel('CO2 Reduction (kg)')
    ax3.set_title('Environmental Impact - CO2 Reduction')
    ax3.grid(True)

    # Cost breakdown for largest cluster
    ax4 = plt.subplot(2, 2, 4)

```

```
largest_cluster_costs = analysis_results['physical_costs'][-1]['cost_breakdown']
labels = list(largest_cluster_costs.keys())
sizes = list(largest_cluster_costs.values())
ax4.pie(sizes, labels
```